

Documentation technique — Ressource Relationnelle

Projet : Ressource Relationnelle

Équipe : PipouTeam

Année : 2024-2025

Sommaire

1. [Présentation du projet](#)
 2. [Choix technologiques](#)
 - [Application web — Frontend](#)
 - [Application web — Backend](#)
 - [Base de données](#)
 - [Application mobile](#)
 - [Outils de travail en équipe](#)
 3. [Architecture des applications](#)
 - [Vue d'ensemble](#)
 - [Architecture frontend](#)
 - [Architecture backend \(API REST\)](#)
 - [Base de données](#)
 4. [Plan de tests](#)
 - [Méthodologie](#)
 - [Cahier de tests](#)
 - [PV de recette](#)
-

1. Présentation du projet

Ressource Relationnelle est une plateforme numérique dédiée au bien-être mental et aux relations humaines. Elle permet à des citoyens, des professionnels de santé et des enseignants de partager et consulter des ressources pédagogiques (articles, vidéos, PDF, podcasts) autour de thématiques comme la communication, la parentalité ou la gestion du stress.

Le projet se compose de deux applications :

- Une **application web** accessible depuis un navigateur, destinée à tous les utilisateurs.
- Une **application mobile** (en cours de développement) pour une utilisation depuis un smartphone.

Les deux partagent la même API (back-end), ce qui évite de dupliquer la logique métier.

2. Choix technologiques

2.1 Application web — Frontend

Le frontend est la partie visible par l'utilisateur dans son navigateur.

Technologie	Rôle	Pourquoi ce choix
Vue 3	Framework JavaScript	Simple à prendre en main, bien documenté, adapté aux projets de taille moyenne
Vuetify 3	Bibliothèque de composants UI	Fournit des composants prêts à l'emploi (boutons, formulaires, tableaux) avec un design professionnel et responsive
Pinia	Gestion de l'état global	Remplaçant officiel de Vuex pour Vue 3, plus simple à utiliser pour stocker des données partagées (ex : l'utilisateur connecté)
Vue Router	Navigation entre les pages	Routeur officiel de Vue, utilisé avec <code>unplugin-vue-router</code> qui génère automatiquement les routes à partir des fichiers dans <code>src/pages/</code>
Vite	Outil de construction	Très rapide au démarrage et lors des modifications, idéal pour le développement

Palette de couleurs : la couleur principale (`#000091`) est inspirée du Design Système de l'État Français (DSFR), ce qui donne une apparence sobre et institutionnelle à la plateforme.

2.2 Application web — Backend

Le backend gère les données et les règles métier. Il se présente sous la forme d'une **API REST** : le frontend lui envoie des requêtes (ex : "donne-moi la liste des ressources") et il répond en JSON.

Technologie	Rôle	Pourquoi ce choix
Node.js + Express	Serveur web	Léger, rapide, et utilise JavaScript comme le frontend — un seul langage pour tout le projet
JWT (JSON Web Token)	Authentification	Système de jetons sécurisés : après la connexion, l'utilisateur reçoit un token qu'il joint à chaque requête pour prouver son identité
bcryptjs	Chiffrement des mots de passe	Les mots de passe ne sont jamais stockés en clair dans la base de données — ils sont transformés en une chaîne illisible
Multer	Gestion des uploads de fichiers	Permet de recevoir des fichiers PDF envoyés par les utilisateurs
Swagger UI	Documentation de l'API	Interface graphique accessible sur <code>/api/docs</code> pour tester les endpoints directement depuis un navigateur
Docker	Conteneurisation	Permet de lancer le backend et la base de données avec une seule commande (<code>docker compose up</code>), sans avoir à tout installer manuellement

Système de rôles : quatre niveaux d'accès sont définis — `citizen` (utilisateur standard), `moderator`, `admin`, `super_admin`. Chaque rôle dispose de permissions différentes sur les ressources.

2.3 Base de données

Technologie	Rôle	Pourquoi ce choix
PostgreSQL	Base de données relationnelle	Robuste, open source, et bien adapté aux données structurées avec des relations entre tables
Docker volume	Persistance des données	Les données sont conservées même si le container est redémarré

Tables principales :

- **users** — les comptes utilisateurs
- **resources** — les ressources publiées (titre, contenu, statut, visibilité)
- **categories** — les thématiques (Communication, Santé, Parentalité...)
- **relation_types** — les types de relations (Soi, Famille, Professionnel...)
- **resource_types** — les formats (Article, Vidéo, PDF, Podcast...)
- **user_resources** — le lien entre utilisateurs et ressources (favoris, consultés)
- **comments** — les commentaires sur les ressources

2.4 Application mobile

L'application mobile est conçue pour fonctionner sur **iOS et Android** à partir d'une seule base de code.

Technologie	Rôle	Pourquoi ce choix
Capacitor	Wrapper natif multiplateforme	Encapsule l'application Vue 3 existante dans une application native iOS/Android, en réutilisant intégralement le code web déjà développé

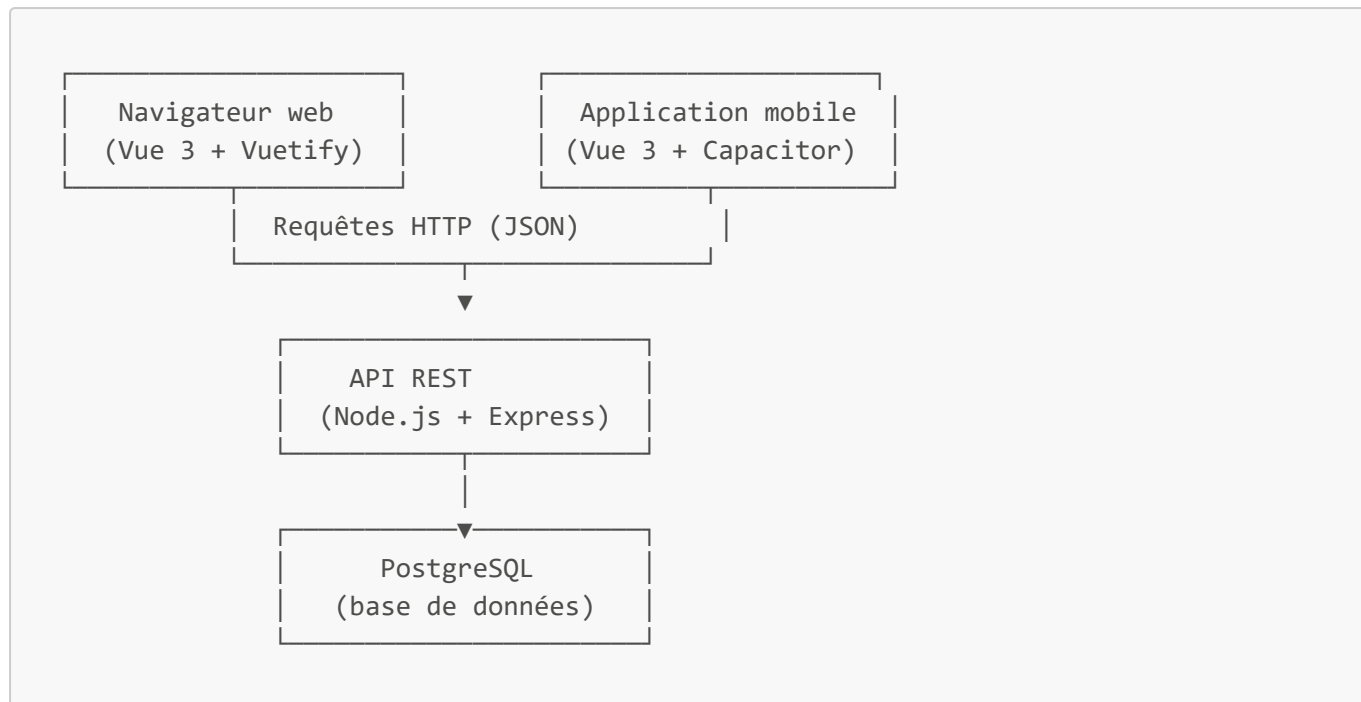
L'application mobile est générée à partir du même code Vue 3 que l'application web, encapsulé par Capacitor. Elle consomme la même API REST. Seule la présentation change (interface adaptée au tactile), les données et règles métier restent identiques.

2.5 Outils de travail en équipe

Outil	Usage
Git / GitHub	Versioning du code et collaboration (branches, pull requests)
GitHub Issues	Suivi des tâches et des bugs
Swagger	Documentation et test de l'API partagée entre développeurs front et back

3. Architecture des applications

3.1 Vue d'ensemble



Les deux applications (web et mobile) communiquent avec **la même API**. Cela garantit la cohérence des données et évite de maintenir deux systèmes distincts.

3.2 Architecture frontend

Le frontend suit le principe d'une **SPA (Single Page Application)** : une seule page HTML est chargée, et la navigation entre les sections se fait sans rechargement complet du navigateur.

```
src/
├── assets/           → Images, logos, fichiers PDF
├── components/       → Composants réutilisables (AppBar, CatalogueCard,
ArticleHeader...)
├── pages/            → Une page = une route (catalogue.vue → /catalogue)
│   └── article/
│       └── [id].vue → Route dynamique (/article/1, /article/2...)
├── stores/           → Données globales partagées entre composants (auth.js)
├── services/         → Communication avec l'API (api.js)
└── router/           → Configuration de la navigation et des gardes de route
```

Gardes de route : certaines pages ([/compte](#), [/mes-ressources](#), [/creer](#)) sont protégées. Si l'utilisateur n'est pas connecté et tente d'y accéder, il est automatiquement redirigé vers [/connexion](#).

3.3 Architecture backend (API REST)

Le backend suit le pattern **MVC** (Modèle - Vue - Contrôleur), adapté à une API :

```
src/
├── controllers/      → Logique métier (ce que fait chaque endpoint)
│   └── authController.js → Inscription, connexion, profil
```

```
├── resourceController.js    → CRUD des ressources
├── progressController.js    → Favoris, consultations
├── routes/                  → Définition des URLs et des méthodes HTTP
│   ├── authRoutes.js       → /api/auth/...
│   ├── resourceRoutes.js    → /api/resources/...
│   └── progressRoutes.js    → /api/progress/...
├── middleware/              → Vérifications appliquées avant d'atteindre un contrôleur
│   └── auth.js              → Vérification du token JWT
├── models/                  → Connexion à la base de données
│   └── database.js          → Pool de connexions PostgreSQL
└── server.js                → Point d'entrée de l'application
```

Fonctionnement d'une requête :

Requête HTTP → Route → Middleware (vérif. token) → Contrôleur → Base de données → Réponse JSON

Principaux endpoints :

Méthode	URL	Description	Auth requise
POST	/api/auth/login	Connexion	Non
POST	/api/auth/register	Inscription	Non
GET	/api/auth/me	Profil utilisateur	Oui
GET	/api/resources	Liste des ressources publiques	Non
GET	/api/resources/:id	Détail d'une ressource	Non
POST	/api/resources	Créer une ressource	Oui
DELETE	/api/resources/:id	Supprimer une ressource	Oui
GET	/api/progress/dashboard	Favoris et consultations	Oui
POST	/api/progress/favorites/:id	Ajouter aux favoris	Oui
POST	/api/upload	Uploader un PDF	Oui

3.4 Base de données

Schéma simplifié des relations :



```
— resource_types
— comments (user_id + resource_id)
```

4. Plan de tests

4.1 Méthodologie

Les tests sont organisés en trois niveaux :

1. Tests manuels fonctionnels

Réalisés directement dans le navigateur par l'équipe de développement. Ils permettent de vérifier que chaque fonctionnalité se comporte comme prévu du point de vue de l'utilisateur.

2. Tests de l'API (Swagger)

Chaque endpoint de l'API est testé via l'interface Swagger accessible sur </api/docs>. On vérifie les réponses en cas de succès et en cas d'erreur (mauvais token, champ manquant, ressource introuvable...).

3. Tests de non-régression

Avant chaque livraison, le parcours utilisateur complet est rejoué pour s'assurer qu'une modification récente n'a pas cassé une fonctionnalité existante.

4.2 Cahier de tests

Module : Authentification

ID	Scénario	Données de test	Résultat attendu	Statut
AUTH-01	Inscription avec des données valides	Prénom : Jean, Nom : Test, Email : jean@test.fr, MDP : Test1234!	Compte créé, redirection vers la connexion	✓ OK
AUTH-02	Inscription avec un email déjà utilisé	Email : citizen@test.com	Message d'erreur "email déjà utilisé"	✓ OK
AUTH-03	Inscription avec un mot de passe trop court	MDP : 123	Message d'erreur "minimum 8 caractères"	✓ OK
AUTH-04	Connexion avec des identifiants corrects	Email : citizen@test.com / MDP : Password123!	Connexion réussie, token stocké, redirection	✓ OK
AUTH-05	Connexion avec un mauvais mot de passe	MDP : mauvais	Message d'erreur "identifiants incorrects"	✓ OK
AUTH-06	Accès à /compte sans être connecté	—	Redirection automatique vers	✓ OK

ID	Scénario	Données de test	Résultat attendu	Statut
			/connexion	
AUTH-07	Modification du profil (prénom/nom/email)	Nouveau prénom : Marie	Profil mis à jour, message de confirmation	✓ OK
AUTH-08	Changement de mot de passe	Ancien MDP correct, nouveau MDP : Nouveau123!	Mot de passe mis à jour	✓ OK
AUTH-09	Changement de mot de passe avec l'ancien incorrect	Ancien MDP : mauvais	Message d'erreur	✓ OK
AUTH-10	Déconnexion	Clic sur "Déconnexion"	Token supprimé, retour à l'accueil	✓ OK

Module : Catalogue et ressources

ID	Scénario	Données de test	Résultat attendu	Statut
CAT-01	Affichage du catalogue	—	Liste des ressources validées et publiques	✓ OK
CAT-02	Recherche par mot-clé	"stress"	Seules les ressources contenant "stress" s'affichent	✓ OK
CAT-03	Filtrage par catégorie	Catégorie : Santé	Seules les ressources de la catégorie Santé	✓ OK
CAT-04	Filtrage par type de relation	Relation : Famille	Seules les ressources liées à la famille	✓ OK
CAT-05	Accès au détail d'une ressource	Clic sur une carte	Page de l'article chargée avec le bon contenu	✓ OK
CAT-06	Affichage d'un contenu texte	Ressource de type texte	Contenu affiché en paragraphes	✓ OK
CAT-07	Affichage d'un PDF	Ressource de type PDF	Visionneuse PDF intégrée	✓ OK
CAT-08	Affichage d'une vidéo YouTube	Ressource avec URL YouTube	Lecteur YouTube intégré	✓ OK
CAT-09	Accès à une ressource inexistante	/article/9999	Message d'erreur "ressource introuvable"	✓ OK

Module : Gestion de ressources (utilisateur connecté)

ID	Scénario	Données de test	Résultat attendu	Statut
RES-01	Création d'une ressource (texte)	Titre + contenu texte	Ressource créée avec statut "en attente"	✓ OK
RES-02	Création d'une ressource (PDF)	Fichier PDF < 20 Mo	Upload réussi, ressource créée	✓ OK
RES-03	Création d'une ressource (YouTube)	URL YouTube valide	Ressource créée avec aperçu de la vidéo	✓ OK
RES-04	Tentative d'upload d'un fichier non-PDF	Fichier .docx	Message d'erreur "seuls les PDF sont acceptés"	✓ OK
RES-05	Création sans titre	Formulaire vide	Validation bloquée, message d'erreur	✓ OK
RES-06	Suppression d'une ressource	Clic sur l'icône supprimer	Dialog de confirmation, puis suppression	✓ OK
RES-07	Annulation de la suppression	Clic sur "Annuler" dans le dialog	Ressource conservée	✓ OK

Module : Favoris

ID	Scénario	Données de test	Résultat attendu	Statut
FAV-01	Ajouter une ressource aux favoris	Clic sur "Ajouter aux favoris" (connecté)	Bouton passe en "Retirer des favoris", ressource ajoutée en BDD	✓ OK
FAV-02	Retirer une ressource des favoris	Clic sur "Retirer des favoris"	Bouton repasse en "Ajouter aux favoris", supprimé en BDD	✓ OK
FAV-03	Bouton favoris invisible si non connecté	Utilisateur non connecté	Bouton non affiché	✓ OK
FAV-04	Affichage des favoris dans /compte	—	Liste des ressources favorites de l'utilisateur	✓ OK
FAV-05	Filtrage des favoris par catégorie	Filtre : Santé	Seuls les favoris de la catégorie Santé	✓ OK
FAV-06	Recherche dans les favoris	Mot-clé : "sommeil"	Seuls les favoris contenant "sommeil"	✓ OK

Module : Navigation et responsive

ID	Scénario	Données de test	Résultat attendu	Statut
NAV-01	Menu desktop visible sur grand écran	Résolution ≥ 960px	Onglets de navigation affichés	✓ OK

ID	Scénario	Données de test	Résultat attendu	Statut
NAV-02	Burger menu visible sur mobile	Résolution < 960px	Bouton hamburger affiché, menu en tiroir	 OK
NAV-03	Onglets "Mon compte" et "Mes ressources" masqués si non connecté	Utilisateur non connecté	Ces onglets n'apparaissent pas	 OK
NAV-04	Accès aux pages légales	Clic sur FAQ, Contact, Mentions légales	Pages affichées correctement	 OK

4.3 PV de recette

Le **procès-verbal de recette** est le document qui atteste que les fonctionnalités livrées ont été vérifiées et acceptées.

Projet : Ressource Relationnelle

Version testée : 1.0

Date de recette : _____

Testeur(s) : _____

Résumé des tests

Module	Nombre de tests	Réussis	Échoués	Taux de réussite
Authentification	10	10	0	100%
Catalogue et ressources	9	9	0	100%
Gestion de ressources	7	7	0	100%
Favoris	6	6	0	100%
Navigation et responsive	4	4	0	100%
Total	36	36	0	100%

Anomalies relevées

ID anomalie	Description	Priorité	Statut
—	Aucune anomalie bloquante relevée lors de cette recette	—	—

Conclusion

Les fonctionnalités testées dans le cadre de cette recette sont conformes aux spécifications.
La version 1.0 de l'application **Ressource Relationnelle** est validée pour mise en production.

Signature du testeur : _____

Date : _____

Documentation rédigée par la PipouTeam — Zone 52, Netherstorm, Azeroth.